creates a Bell state, one of the simplest entangled states in quantum mechanics. First, we apply a Hadamard gate to one qubit to put it into a superposition. Then, we add a CNOT gate to entangle this qubit with a second qubit, linking their states. This creates a Bell state, where the two qubits are in an entangled state, meaning their outcomes are correlated.

Optimisation techniques in Pytket

Advanced optimisation strategies can be leveraged in Pytket to minimise gate counts and circuit depth, enhancing the performance and reliability of quantum circuits on real hardware.

Circuit optimisation strategies: Pytket is renowned for its robust circuit optimisation capabilities, employing strategies like gate cancellation and commutation to streamline quantum circuits. Gate cancellation involves removing redundant gates that don't alter the outcome, while commutation reorders gates without changing the circuit's functionality. These strategies are essential, particularly for hardware-limited quantum devices where resource constraints are tight. By reducing unnecessary operations, Pytket helps conserve quantum resources, making circuits more efficient and enhancing their suitability for execution on real quantum hardware.

Minimising gate count and circuit depth is crucial in quantum computing, as each additional gate increases the probability of decoherence and other errors. Pytket's optimisation suite addresses this by automatically identifying and removing extraneous gates, shortening the circuit and reducing the computational load. This improvement in circuit fidelity leads to more accurate results, especially important when running complex quantum algorithms. With reduced gate count and circuit depth, Pytket allows developers to design circuits that are not only optimised for performance but also better suited for current quantum hardware capabilities.

Examples of circuit optimisation in action: Consider a scenario where we run the same circuit, both unoptimised and optimised, on backends like IBM Q or Google Cirq. In the unoptimised version, the circuit may contain redundant gates, leading to longer execution times and a higher likelihood of errors. By applying Pytket's optimisation techniques, we can observe a tangible reduction in execution time and an improvement in accuracy. These real-world examples highlight the practical impact of Pytket's optimisations, demonstrating how streamlined circuits lead to enhanced performance on quantum devices.

Running quantum circuits on different backends

Quantum circuits can be seamlessly executed on a variety of platforms, including IBM Q and Rigetti, with Pytket's intuitive backend integration, enabling easy benchmarking and performance comparisons.

Supported quantum hardware and simulators: Pytket is compatible with a variety of quantum hardware platforms, including IBM Q, Rigetti, Google Cirq, and several local simulators. This support provides developers with the flexibility to test their circuits on different types of hardware, each with unique strengths. For instance, IBM Q's robust infrastructure is well-suited for general-purpose circuits, while Rigetti's architecture is often preferred for certain gate configurations and hybrid algorithms. Local simulators are also available for rapid testing and debugging before deployment to quantum hardware, making Pytket a versatile choice for different stages of quantum circuit development.

Working with IBM Q, Rigetti, Google Cirq, and more: While each quantum backend has specific requirements and capabilities, Pytket abstracts these details, allowing users to focus solely on circuit design and optimisation. The toolkit automatically adapts the circuit format, gate sets, and configurations to fit the selected backend, sparing developers from manually adjusting their code for each platform. This backend flexibility is particularly useful for researchers who need to test the same algorithm on multiple devices, as they can switch between IBM Q, Rigetti, and Cirq with minimal effort.

An example of running a circuit on IBM Q with Pytket: To illustrate, consider setting up and running a basic Bell state circuit on IBM Q. First, we construct a Bell state circuit in Pytket by applying a Hadamard gate followed by a CNOT gate to entangle two qubits. Once the circuit is defined, Pytket provides a straightforward syntax for connecting to the IBM Q backend and submitting the job. This example walks users through the steps, from building the circuit to observing results, demonstrating how Pytket simplifies execution on IBM Q and similar quantum platforms.

Classical control flow and hybrid algorithm support

Complex workflows can be implemented with Pytket's support for classical control flow, enabling the development of hybrid algorithms that integrate quantum computations with classical optimisation techniques.

Conditional operations and control flow: Pytket includes support for classical control flow, enabling conditional operations in quantum circuits. This functionality allows a circuit to adapt based on the results of intermediate measurements, which is essential for implementing advanced algorithms and handling error mitigation. For example, if a particular qubit measurement yields a specific result, the circuit can take an alternate path, adjusting subsequent operations accordingly. This kind of control flow is vital for quantum-classical hybrid algorithms, where classical computations may guide the quantum